Onyx: Jet Browser, the WebKit port

Onyx · multi-process OS on Raspberry Pi 4

2026-10-04: this browser is Jet. The WebKit browser was first called **Web** (`user/Apps/jet`, the package `web`, `SD:/apps/web.app`) beside the NetSurf port named Jet. NetSurf's Jet, its libraries, its PC bench and its Windows build were removed, and this browser took the name: **`user/Apps/jet`, the package `jet` (2.0), `SD:/apps/jet.app`, `run jet`**. Below, "Web" is its name at the time each part was written; what keeps the old name on purpose: its lines in the kernel log (`web: …`, `webview: …`), the switch files (`SD:/etc/web-*`), the script `tools/webkit/build-web.sh` and its output `$BUILD/bin/web`. The user agent is WebKit's own.

2026-10-05 -- the layout of `user/` changed (docs/03 section 5): `user/gpucomp` is `user/Libs/gpucomp`, `user/av` is `user/Libs/av`, `user/libc/posix` is `user/Runtime/libc/posix`, the kits are in `user/Kits`. The patch series says the new paths (`ONYX_GPUCOMP_INCLUDE_DIR`, `ONYX_AV_INCLUDE_DIR` in WebKit's CMake). **A WebKit tree patched before that date, built against an Onyx clone you then update, must have these two CMake lines follow** (edit them in the tree, or re-apply the series), and the sysroot's POSIX library is reinstalled from its new place: `make -C user/Runtime/libc/posix install PREFIX=aarch64-onyx-elf-SYSROOT=...`.

Status / how to resume (2026-10-03)

Now: Web, the browser on WebKit2, runs on the Pi (see "Step 3" below: kotonstudio.com over HTTPS in about 3.2 s), and **the JavaScriptCore JIT runs there** (the Baseline JIT and the DFG on kapi v78's `PROT_EXEC`: "The JIT" below). What follows was written at step 1 and stays true for `jsc`.

Step 1 (WTF + JavaScriptCore → the `jsc` shell) is done: it passes on the PC bench and on the Pi (the five steps of the Pi test below), and is in `main`. Step 2 (WebCore) is next, on a branch `webkit-port`. `/bin/jsc` (the LLInt and WebAssembly's interpreter, no JIT; 33.1 MB with ICU's data) is staged in `sdcard/`, with `SD:/docs/jsc/smoke.js` and `bench.js`.

Pinned WebKit revision: `b8a7a626127c0010a557c9d6466fefd38d9477c1` (WebKit `main`, 2026-10-02; its `Source/ThirdParty/skia` is Skia m154 `588b550a`, the copy already ported into the sysroot: `third_party/skia-m154/README.onyx`). Set in `tools/webkit/revision.sh`.

What is there

- **The scripts** (`tools/webkit/`): `fetch.sh` (a sparse, shallow checkout of the pinned revision outside the repo — default `/home/user/webkit`, else `$HOME/webkit`, `WEBKIT_DIR=` to choose — about 280 MB: the top-level CMake files, `Source/cmake`, `Source/WTF`, `Source/JavaScriptCore`, `Source/bmalloc`, `Tools/Scripts/webkitperl`, `JSTests/stress`, `JSTests/es6`; then `git am` of the patch series on a branch `onyx`), `export-patches.sh` (the branch `onyx` → `tools/webkit/patches/`), `build-jsc.sh` (configure + build `jsc`; `INTERP=llint` (default: the LLInt, with WebAssembly) or `INTERP=cloop` (the C++ interpreter, without), each in its own build tree `<WEBKIT_DIR>-build/jsc-<interp>`; `install` strips it into `user/BinUtils/jsc.elf`), `test-jsc.sh` (the tests on the posixsim bench, below), `smoke.js`, `bench.js`. `make -C user/BinUtils jsc` = `fetch.sh` + `build-jsc.sh install`.
- **The port** (the patch series, `tools/webkit/patches/`):

1. `0001` — `PORT=Onyx`: `Source/cmake/OptionsOnyx.cmake` (static libraries; `ENABLE_JIT` OFF, `ENABLE_C_LOOP` ON by default, WebAssembly / sampling profiler / remote inspector OFF, `USE_SYSTEM_MALLOC` ON, no mimalloc / IsoMalloc, `USE_64KB_PAGE_BLOCK`, the generic event loop), Onyx in `WebKitCommon.cmake` (`CMAKE_SYSTEM_NAME Onyx` → `WTF_OS_ONYX` + `WTF_OS_UNIX`), `OS(ONYX)` in `wtf/PlatformOS.h` (from `__onyx__`, a Unix), `CeilingOnPageSize` 64 KB, `wtf/PlatformOnyx.cmake` (POSIX + Unix sources, `RunLoopGeneric`, `WorkQueueGeneric`, `MainThreadGeneric`, `MemoryPressureHandlerUnix`), `wtf/onyx/` (the process's memory status from libonyxposix's `getrusage`), `bmalloc/PlatformOnyx.cmake`, `JavaScriptCore/PlatformOnyx.cmake`.
2. `0002` — WTF builds: `-D_GNU_SOURCE` (newlib hides its POSIX / GNU declarations from a strict `-std=c++NN` compile), **no machine context and no thread suspension** (see below), the signal handlers never installed, the memory-pressure hold-off timer and memory status for `OS(ONYX)`, the thread name through `pthread_setname_np(pthread_self(), name)`.
3. `0003` — JavaScriptCore builds and `jsc` links: the bytecode cache version from the build's timestamp (no `<link.h>` in a static program), ICU's static archives in the order i18n, uc, data.
4. `0004` — the **LLInt**: offlineasm's `globaladdr` by direct `adrp` / `add :lo12:` (static, non-PIC: no GOT), `OS(ONYX)` among `InlineASM.h`'s ELF platforms, `cacheFlush` through `__builtin___clear_cache` (not reached without a JIT).
5. `0005` — the collector: the mutator **finishes the collections before it releases heap access** (see below).
6. `0006` — **WebAssembly** in its in-place interpreter (IPInt), and the LLInt as the port's default: `ENABLE_C_LOOP` OFF, `ENABLE_WEBASSEMBLY` ON, the BBQ / OMG JIT tiers OFF; the fault handler and "fast" memories off for `OS(ONYX)` (explicit bounds checks), `collectContinuously` forced off.
7. `0007` — WebAssembly exceptions caught by WebAssembly code in a build without the JIT (`genericUnwind` had no catch routine for an IPInt handler when `ENABLE(JIT)` is off: every `catch` / `try_table` ended the process). Not specific to Onyx: worth sending upstream. New WebKit-side files carry WebKit's BSD-2 header with "Onyx contributors"; WebKit's LGPL / BSD notices are untouched.

- **libonyxposix** (commits on the branch): `<uchar.h>`, `onyx-cc.specs` (`__unix__`, `-pthread` accepted), `MAP_FILE` in `<sys/mman.h>`.

Two things Onyx does differently (and why they matter for the next steps)

- **No asynchronous signals, so no thread suspension.** On Onyx a signal handler only runs when the process raises the signal itself; a faulting process is ended by the kernel; newlib's `struct sigaction` has no `sa_sigaction`. WTF suspends a thread with a signal (`Thread::suspend`, `ThreadingPOSIX.cpp`) and JavaScriptCore uses that for three things: the sampling profiler and the JIT tiers' signal-based VM traps (both off here), and **the collector scanning the stack of a thread that has no heap access**. `Thread::suspend()` is a fatal error for `OS(ONYX)`; `PlatformRegisters` is the stack pointer only (`HAVE(MACHINE_CONTEXT)` off).
- **The collector never scans another thread's stack.** A thread with heap access scans its own stack (stopped at a safepoint, or conducting the collection itself). Without it — the VM's lock released, the thread back in its run loop — an unfinished collection goes on in the collector thread, which must suspend the thread to read its stack: on Onyx that ended the process as soon as a program allocated, then waited (`setTimeout` in `jsc`; any page in a browser). Patch `0005`:

`Heap::releaseAccess()` first waits for the outstanding collections (`finishCollectionsBeforeReleasingAccess`). No collection starts while the mutator has no access (requests come from the thread holding the VM's lock). Cost: the end of a collection is a pause on the mutator when it goes idle; marking stays concurrent and parallel while it runs. **Limits: one thread per VM** (a VM shared between threads would need the suspension; workers have their own VM), and no `--collectContinuously` (a debugging option whose own thread asks for collections while the mutator may have no access: forced off). A kernel call "suspend this thread and give me its registers" would lift both (a kapi addition: not needed so far).

- **WebAssembly without faults.** JavaScriptCore normally lets an out-of-bounds access fault in a guard region and recovers in a signal handler. Not possible on Onyx: memories are bounds-checked explicitly (an option JavaScriptCore has). What that costs: **no shared memories** (the parser ties them to the fault handler: "shared memory is not enabled") — so no WebAssembly threads — and **no SIMD** (the parser only has it with the B3 JIT). Everything else of WebAssembly runs in IPInt: MVP, exceptions, tail calls, GC, references, multi-memory, memory64.

Tests (the PC bench: `sh tools/webkit/test-jsc.sh`, qemu-user on `tools/tests/posixsim`)

`jsc` is relinked against the bench's libonyxposix + fake kapi and run under qemu. `INTERP=llint|cloop`, the suites `smoke es6 stress bench`, `STRESS_STEP=1` for every stress test.

Suite	LLInt (+ WebAssembly)	C_LOOP
`smoke` (`smoke.js`: 34 checks with WebAssembly, 29 without — the language, ICU, the collector, WebAssembly, microtasks and timers; `jsc -e`; exit 3 on an exception)	pass	pass
`es6` (WebKit's `JSTests/es6`, 605 tests: 595 expected to pass, 10 to fail)	595 + 10 as expected	595 + 10 as expected
`stress` (WebKit's `JSTests/stress`: the 4690 tests the harness runs with its default options)	4688 pass, 2 fail as expected	4686 pass, 4 fail as expected
`wasm` (WebKit's `JSTests/wasm`: the directories of its "WebAssembly suite", 611 tests after the ones with a `//@ skip` or a run command of their own)	594 pass, 17 fail as expected	— (no WebAssembly)

The expected failures, each with its reason in `test-jsc.sh`: in `stress`, `intl-relativetimeformat.js` and `string-localeCompare.js` (Swahili and Ewe are not in the filtered ICU data, `tools/ports/icu/data-filter.json`), and with C_LOOP the two tests that use WebAssembly; in `wasm`, 5 tests that need SIMD and

12 that need shared or "signaling" memories (above). The harness's options and time zone are reproduced (`--maxPerThreadStackUsage=1572864`, `TZ=US/Pacific`...). The stress tests with other `//@` directives (JIT tiers, special options: about 1150) are not run.

`bench.js` under qemu on the PC (not the Pi's timings; qemu flatters the C++ loop):

	LLInt	C_LOOP
`fib(30)`	737 ms	1062 ms
20M additions in a loop	2659 ms	3722 ms
1M object allocations	683 ms	817 ms
sort 300k numbers	1599 ms	2417 ms
5M typed-array doubles	6142 ms	7550 ms
regexp, 100k matches	359 ms	373 ms

The LLInt is the one staged: it needs no executable memory at run time (it is assembled at build time), is the faster, and is the one WebAssembly works with. `el0scan` on the unstripped `jsc`: clean (libgcc's guarded SME helpers only, as every program of the toolchain).

The Pi test (2026-10-02: the five steps pass)

On a card made from the branch's `sdcard/` (kernel v76 or later), in the Terminal:

1. `jsc -e "print(6*7)"` → `42`.
2. `jsc SD:/docs/jsc/smoke.js` → the last line is `smoke: ok (34 checks)` (WebAssembly included).
3. `jsc SD:/docs/jsc/bench.js` → nine lines of timings (send them back: the first real numbers).
4. `jsc` alone → a `>>>` prompt: `1+1`, `new Intl.DateTimeFormat("fr-FR", {dateStyle: "full"}).format(new Date())`, `quit()`.
5. `jsc -e "a=[];for(i=0;i!=300000;i++)a.push({i});setTimeout(function(){print('late')},2000)"` → `late` after two seconds (the collector while the shell waits; on the Pi 3 to 4 seconds in all: the 33 MB image loaded from the card, JavaScriptCore's start, the loop, then the timer's two seconds). Written without `<`, `>`, `|` or spaces because the shell had no quoting then (a first version, with `i < 300000` and arrow functions, answered "cannot open input file" without starting `jsc`); since 2026-10-02 `/bin/cmd` honours `"..."`, `'...'` and `\` (docs/04 §7 *Quotes and escapes*), so `jsc -e "let a = []; for (let i = 0; i < 300000; i++) a.push({i}); setTimeout(() => print('late'), 2000)"` works as well.
6. Memory: the Task Manager's figure for `jsc` while step 3 runs. `bench.js` on the Pi 4 (the LLInt, 2026-10-02) — 2 to 6 times the PC's qemu figures above:

	Pi 4
`fib(30)`	1852 ms
20M additions in a loop	5595 ms
1M object allocations	2085 ms
300k string concat / split	2307 ms
sort 300k numbers	4896 ms
regexp, 100k matches	1330 ms
Map, 500k set / get	4001 ms

	Pi 4
5M typed-array doubles	17357 ms
JSON, 20 × 200 KB	1609 ms

What can differ from the bench: the loader on a 33.1 MB image, the kernel's `vm_*` under the collector's reservations, the main thread's stack size, timing.

Resume

From a fresh clone, in one command: `sh tools/build-jet.sh` [`--no-jit`] [`--jobs N`] (Linux x86_64 or WSL 2). It builds Onyx first (`tools/build-sdcard.sh`), fetches the `aarch64-onyx-elf` toolchain, installs the POSIX sysroot, builds the libraries from `third_party/` (`tools/ports/build-all.sh`), fetches WebKit at the pin with the patch series (`fetch.sh`, `TESTS=0`), builds WebKit with the JIT as the package is built (`BUILD=<WEBKIT_DIR>-build/webkit-jit`, `CMAKE_EXTRA="-DONYX_WEBKIT=ON -DENABLE_JIT=ON -DENABLE_DFG_JIT=ON -DENABLE_FTL_JIT=OFF"`, `TARGET=WebKit`), then Jet (`build-web.sh` → `sdcard/apps/jet.app/main`). Each step is skipped when done: a second run continues. The host packages it asks for: `git build-essential python3 perl ruby gperf unifdef cmake ninja-build curl xz-utils bison flex pkg-config` (+ `ccache`). The steps by hand:

```
sudo apt-get install -y gperf unifdef cmake ninja-build ruby ccache   # host tools (docs/LOCAL-AGENT-WEBKIT.md)
sh tools/toolchain/fetch.sh                                  # aarch64-onyx-elf into /opt/toolchains
make -C user/Runtime/libc/posix install PREFIX=aarch64-onyx-elf-   # the sysroot (out/sysroot-onyx)
sh tools/ports/build-all.sh icu                              # ICU into the sysroot (webkit: all of them)
sh tools/webkit/fetch.sh                                     # the checkout at the pin + the patches
sh tools/webkit/build-jsc.sh install                         # -> user/BinUtils/jsc.elf (7 min on 16 cores)
STRESS_STEP=1 sh tools/webkit/test-jsc.sh smoke es6 stress wasm bench   # 20 min
# a change: commit in the checkout (branch onyx), then: sh tools/webkit/export-patches.sh
```

(`ninja -C <build> -k 0 jsc` lists many errors at once. WebKit's CMake uses ccache when it is installed.)

Next steps

1. Step 2, WebCore (below), on the branch `webkit-port`. The package `jsc` 1.0.0 is published.
2. For later, in the kernel: PROT_EXEC for a JIT, a call to suspend a thread (above), a malloc that returns memory, a streaming ELF loader and shared code pages (a static WebKit is 60–100 MB).

Step 2: WebCore (2026-10-02, branch `webkit-port`)

Where it stands: WebCore builds for Onyx, links, and renders a page — on the PC bench and on the Pi (2026-10-02). On the Pi 4, `wctest SD:/docs/page1.html RAM:/page1.png` (the 80 MB program copied by hand to `SD:/bin`) printed the same title, element count and text as the bench — the line written by the page's script included — and wrote the same picture (47 465 bytes, as on the bench; opened in the Image Viewer): the kernel's loader takes an 80 MB static image, the card's fonts are found, Skia paints on the CPU, JavaScriptCore runs inside WebCore. **5 to 6 seconds from the command to the prompt** (the user's timing): most of it is probably the loader reading and copying the 80 MB image from the card (`jsc`, 33 MB, takes 1 to 2 seconds before it runs anything); WebCore's own work is one second under qemu on the PC. Not broken down yet: `wctest` could print its own times. What it means for step 3:

WebKit2 starts several processes of large images — the streaming ELF loader and the code pages shared between processes of one executable (docs/POSIX-PLAN.md §5, §13) will matter there. `libPAL.a` and `libWebCore.a` build (`sh tools/webkit/build-webcore.sh`: 30 minutes on 16 cores for a first build, 18 to 22 GB of memory at 14 jobs); `wctest` (`tools/webkit/wctest.cpp`, built by `build-wctest.sh`: WebCore alone, the "empty" clients, no window and no network) loads an HTML file, lays it out, paints it with Skia on the CPU into a PNG; `sh tools/webkit/test-webcore.sh` runs it under qemu on `tools/webkit/tests/page1.html` and checks what it prints and the picture's pixels: **21 checks pass** — the title and the text (UTF-8, the card's fonts through `SkFontMgr_onyx`), CSS boxes, flexbox, grid, a gradient with rounded corners, an inline SVG, form controls (WebKit's Adwaita theme), a script that changes the DOM and draws into a canvas (JavaScriptCore inside WebCore). One second under qemu. The program is 80 MB (58 MB of code, 22 MB of data with ICU's): not staged on the card (too big for the repository) — for a Pi test, strip `<build>/bin/wctest` and copy it by hand.

Patches `0008`–`0011` (`tools/webkit/patches/`): `USE(SKIA_GPU)` (Skia's GL / Ganesh paths compiled out: 24 files), the curl back end and PAL's digests on mbedTLS, WebCrypto's containers and stubs for mbedTLS, and the port itself (`OptionsOnyx.cmake` with `ONYX_WEBCORE`, `PlatformOnyx.cmake`, the Onyx files). What was found on the way, beyond the study below:

- **The theme is WebKit's own Adwaita** (`USE_THEME_ADWAITA`, as WPE, GTK and Windows): drawn form controls and scrollbars with no GLib and nothing from the system. No Onyx theme was written.
- **The libwpe pasteboard and editor files call no libwpe function**: Onyx shares them (`PLATFORM(ONYX)` beside `USE(LIBWPE)`); the real clipboard is behind the `PasteboardStrategy` (step 3, on `clipd`).
- WebCore needs a `LoaderStrategy` even to make a page (`Page::firstTimeInitialization`); the "empty clients" page is fully sandboxed (scripts refused) until its sandbox flags are cleared: both are in `wctest.cpp`, and are WebKit2's job in step 3.
- The link asked for 36 symbols in all (pasteboard, editor, WebCrypto, one keyboard function).

Not done / not tested yet in step 2: the memory on the Pi was not measured, the time only as a whole; **the network path is compiled, not exercised** (curl with the mbedTLS glue of `MbedTLSHelper.cpp`: a page loaded over HTTPS comes with step 3's network process, or a `wctest` with a real loader); the public suffix list is an interim rule (`PublicSuffixStoreOnyx.cpp`); WOFF2 fonts, video and audio are off; `PlatformScreenOnyx` answers 1920 × 1080.

WebCrypto on mbedTLS (patch `0012`; the user's choice, 2026-10-02: one crypto stack rather than OpenSSL's libcrypto beside it). `Source/WebCore/crypto/mbedtls/`, on mbedTLS's classic API: HMAC, AES-CBC / CTR / GCM / KW (CFB-8 too, which this WebKit revision refuses from script on every port), PBKDF2, HKDF, ECDSA and ECDH on P-256 / P-384 / P-521, RSASSA-PKCS1-v1_5, RSA-OAEP, RSA-PSS; the keys' raw, JWK, SPKI and PKCS#8 forms are parsed and written there (mbedTLS's own parser accepts more than WebCrypto allows, and it has no PKCS#8 writer). mbedTLS is built without threading and its RSA / EC operations write into the context, while WebCrypto runs on work queues: **every operation works on a private copy of the key**. RSA key generation runs on a work queue. Nothing was added to the mbedTLS port. Not there: Ed25519 / X25519 (as the OpenSSL back end); the wrapping of serialised keys passes them through (as the OpenSSL back end). `sh tools/webkit/test-webcrypto.sh` runs `tests/crypto1.html` in `wctest` under qemu (`WCTEST_WAIT_TITLE`: the run loop turns until the page's title says it is done): **299 checks pass** — published vectors (FIPS 180-4, RFC 2202 / 4231 / 3394 / 5869 / 5903 / 6070 / 6979 / 7515 / 7914, SP 800-38A, the GCM specification), values and DER encodings checked against OpenSSL (`tests/gen_crypto_vectors.py`, which made the page's vectors), and refusals

(corrupted tags and signatures, points off the curve, truncated DER). Not run on the Pi. **To know**: the keys, nonces and salts generated on the Pi are as good as the kernel's random source, today a tick-seeded software generator (`onyx_entropy.c`) — to be made a real one before the browser is used for anything that matters; mbedTLS's AES is table-based (hardware AES is off in the port): not constant-time with respect to the cache.

The design (the study it started from)

Decided (the user, 2026-10-02): the graphics, in this order — (1) a browser rendered on the **CPU, without GL** (Skia raster); (2) once that has worked well for a few days, a **compositor on the V3D** (Onyx's own kernel driver, as Jet with `user/Libs/gpucomp`); (3) **WebGL is a goal for later**, and its route is chosen: **an ANGLE back end of our own that drives Onyx's V3D kernel driver directly, with only Mesa's shader compiler library (SPIR-V → NIR → QPU) behind it** — not a port of the whole of Mesa (no Gallium `v3d` driver, no emulation of Linux's DRM interface, no Mesa EGL): the fewest interfaces, the most direct path. (WebKit's WebGL is ANGLE, which translates GLSL to another shader language, never to QPU code: Mesa has the only GLSL → QPU compiler. The GL state → control lists driver is then ours to write. A software WebGL would need executable memory.) To study before any code: ANGLE's back-end interface, Mesa's compiler built alone. Nothing done now should close that door: GL is cut out of the build by a flag, not removed.

What a study of the pinned sources found (read, not compiled: verify when building):

- **No port builds WebCore without GL.** Windows, WPE / GTK and PlayStation all composite with GL (TextureMapper or Coordinated Graphics); `platform/graphics/skia/SkiaCompositingLayer*` is GL-only too. With `USE(SKIA)`, about ten files use Ganesh / GL whatever the configuration: `PlatformDisplaySkia.cpp`, `ImageBufferSkiaAcceleratedBackend.cpp`, `SkiaGPUAtlas.cpp`, `SkiaReplayAtlas.cpp`, `SkiaReplayCanvas.cpp` (whole files), and hunks of `GraphicsContextSkia.cpp`, `NativeImageSkia.cpp`, `ImageUtilitiesSkia.cpp`, `SkiaUtilities.cpp/.h`, `SkiaSerializedImageBuffer.cpp`, `SkiaRecordingResult.h`, `platform/graphics/ImageBuffer.cpp`, `PlatformDisplay.h/.cpp`. The run-time paths are already CPU-safe (`RenderingMode::Unaccelerated`): the problem is compiling and linking. **Chosen: a guard patch** (a flag such as `USE(SKIA_GPU)`, set by the other ports, unset by Onyx) rather than building Ganesh with stubbed GL. Onyx's Skia (`tools/ports/skia`) has no Ganesh.
- **Compositing compiled out**: none of `USE_TEXTURE_MAPPER`, `USE_COORDINATED_GRAPHICS`, `USE_GRAPHICS_LAYER_WC`, `USE_LIBEPOXY`, `USE_ANGLE_EGL`, `USE_LIBWPE`; `ENABLE_WEBGL`, `ENABLE_GPU_PROCESS`, `ENABLE_ASYNC_SCROLLING`, `ENABLE_VIDEO`, `ENABLE_WEB_AUDIO` OFF for the first build. Onyx supplies `GraphicsLayer::create()` (a `GraphicsLayerOnyx` with no-op `setNeedsDisplay*`: the only pure virtuals); pages paint through `LocalFrameView` into a plain `GraphicsContext` with `acceleratedCompositingEnabled` false at run time. WebKit2 has the matching path (`DrawingAreaCoordinatedGraphics::display` into a `ShareableBitmap`, `UIProcess/skia/BackingStoreSkia.cpp`): step 3.
- **CMake**: `Source/CMakeLists.txt` adds `ThirdParty/skia` when `USE_SKIA` (Onyx: an imported `Skia::Skia` from the sysroot, with a `<skia/...>` include directory — the sysroot has `include/skia/include/...`); `WebCore/CMakeLists.txt` links GLES / EGL for every port without epoxy or ANGLE (to skip for Onyx).
- **Fonts**: `FontCacheSkia.cpp` and `SkiaSystemFallbackFontCache.cpp` call fontconfig directly except for Android / Windows: Onyx takes that side of the guards with `SkFontMgr_New_Onyx`.
- **TLS**: the curl back end uses OpenSSL in `OpenSSLHelper.cpp` (the certificate chain and its summary), `CurlSSLVerifier.cpp`, `CertificateInfoCurl.cpp`, a few places of `CurlContext.cpp`.

curl 8.16's mbedTLS back end already gives what they need (`CURLOPT_SSL_CTX_FUNCTION` with the `mbedtls_ssl_config *`, `CURLINFO_TLS_SSL_PTR`, CA blobs): **mbedTLS glue, about 600 lines** (docs/POSIX-PLAN.md §5.7's choice). Read the chain after the handshake rather than replacing curl's own verify callback.

- **WebCrypto cannot be switched off** (no option; its sources are unconditional) and its only portable back ends are OpenSSL (`crypto/openssl/`, 2700 lines) and gcrypt. **First build: a stub back end** (every `platform*` function answers NotSupported; SHA digests through `PAL::CryptoDigest` on mbedTLS). The decision was between a real back end on mbedTLS (one crypto stack, about 3000 lines of ours to maintain) and OpenSSL's libcrypto ported for it (Apache-2.0, WebKit's code unchanged, a second crypto library on the card): **mbedTLS, done** (above).
- **Others**: libpsl (public suffixes) is required by `Curl.cmake`: a `PublicSuffixStoreOnyx.cpp` first, the library (MIT) later — cookies and site isolation need a real list. `USE_WOFF2` OFF first (FreeType with brotli, or libwoff2, later). No libwpe: `PasteboardOnyx`, `PlatformPasteboardOnyx` (on `clipd`), `PlatformKeyboardEventOnyx` instead of the `libwpe/` files.
- **The per-port files** (the PlayStation ones are 40–160 lines each): accessibility (`AXObjectCache`, `AccessibilityObject`: no-ops), `SystemFontDatabase`, `CurlSSLHandle` (the CA bundle `SD:/res/ca-bundle`), `NetworkStateNotifier`, `MIMETypeRegistry`, `PlatformScreen`, `ScrollbarTheme`, `Theme`, `UserAgent`, `RenderTheme`; `PLATFORM(PLAYSTATION)` appears only 13 times in WebCore (`RenderTheme.h`, `PlatformRenderTheme.h`, `AXCoreObject.h`, `ChromeClient.h`, `EmptyClients.h`...): an `ONYX` branch beside each that matters. PAL: `CryptoDigestMbedTLS.cpp`.

The order of work: `OptionsOnyx.cmake` (`ENABLE_WEBCORE`, the packages, `USE_SKIA`, `USE_CURL`, `USE_HARFBUZZ`, `USE_MBEDTLS`), `WebCore/PlatformOnyx.cmake` and `PAL/pal/PlatformOnyx.cmake`, the CMake patches, then `ninja -k 0` on WebCore and the errors in families (as step 1), the stubs first, the real implementations (TLS glue, pasteboard, theme) once it links. The target of step 2 on the PC: WebCore links into a test program that loads a page from a file and paints it into a PNG.

Step 3: WebKit2 and Web, the browser (2026-10-03)

Web runs on the Pi (`user/Apps/jet`, package `web` 93 MB; docs/04 *Web*): the window on uikit, the page drawn by WebKit2's three processes — **one program** (`apps/jet.app/main`) that is the UI and, started again with `--onyx-webkit-process=web|network`, the web and the network process; the kernel maps the program's image once for all (`image …: shared in 0 ms` in kmsg). Tried there (overnight, over telnet and VNC, the user's Pi 4 8 GB): the start page (a `file:` page, 1.3 s from the start); **kotonstudio.com over HTTPS** with its redirection, web fonts, images, CSS animations — about **3.2 s from the address typed to the load's end** (`web: [t]` lines), its title in the window's frame; the wheel scrolling to the end and back; Back / Forward; typing an address; a `target=_blank` link opening a **new window** (a new launch, the user's choice); GitHub's React pages (about 50 s for a repository's page: the network answers in 1–3 s a file, the rest is JavaScript in the interpreter — the JIT is roadmap step 3).

What was written: patches `0013` (WTF), `0014` (the port: `PlatformOnyx.cmake`, `ONYX_WEBKIT`, the software drawing path behind `USE(NON_COMPOSITED_DRAWING_AREA)`, the IPC monitor thread, the launcher, the C API with the Onyx view — `docs/WEBKIT2-STUDY.md` was the plan), `0015`; the browser (`main.cpp` the window, `engine_webkit.cpp` on the C API, `engine_mock.cpp` for the desktop simulator, where the window is photographed); `tools/webkit/build-webkit.sh`, `build-web.sh`, `wk2test.cpp` + `build-wk2test.sh` + `test-webkit.sh` (the bench: the three processes load a page by `data:`, `file:` and `http:` and paint it — all checks pass).

What was found on the way, and fixed:

- **WTF's RedBlackTree::remove left the removed node's links** (CheckedPtrs): the generic RunLoop's stopped timers kept counts on their neighbours, whose destruction then crashed (`0013`).
- **curl was built without its multi wake-up** (`CURL_DISABLE_SOCKETPAIR`): WebKit's request scheduler polls without a limit and waits to be woken — no request ever started (`tools/ports/curl/build.sh`: the wake-up is a pipe).
- **The IPC monitor thread read on its own thread** (PlayStation's design) beside the connection's queue: it now only waits for data and has the queue read it (`0015`).
- **A heap corruption in Skia's glyph painting, on the Pi only** (the web process killed while scrolling kotonstudio.com: `_malloc_usable_size_r` from `SkContainerAllocator`, a chunk whose size field held a pointer). Found with two tools written for it: the kernel now prints a killed program's **backtrace** (the frame-pointer chain, `el0: … backtrace:`), and `tools/webkit/heapcheck.c` (a checking malloc in front of newlib's: canaries, a quarantine filled and checked; `HEAPCHECK=1 sh build-web.sh`). What stopped it: Skia no longer grows its containers to newlib's `malloc_usable_size` (`tools/webkit/skmallocsize.c`, linked by `build-web.sh`). **newlib's usable size is not the fault** (checked 2026-10-03 with `/bin/malloctest`, docs/03 §5.4: blocks from `malloc` / `calloc` / `memalign` / `posix_memalign` / `aligned_alloc` / `realloc` filled to `malloc_usable_size` and re-read, next to the top of the heap while newlib trims it with a negative `sbrk`, across the program's own `sbrk` calls (the fenceposts), at random and in four threads: no byte of a block changed, no header damaged, on the bench and on the Pi with the real `CAddressSpace::Sbrk`; the same test told a usable size 8 bytes too large is caught at the first block). Read against the source (newlib 4.4 `_mallocr.c`, `MALLOC_ALIGNMENT` 16, `SIZE_SZ` 8): the usable size is the chunk less one size word, as in every dlmalloc; `memalign`'s leading and trailing splits, `malloc_extend_top`'s fenceposts and `malloc_trim` keep it true. So there is nothing to correct in libonyxposix, in the kernel's `Sbrk` or in the toolchain for it, **and the cause of the crash is still unknown**: the workaround changed what Skia writes and where every block lies, which hides a corruption as well as it cures one — it stays linked until the cause is found. What the crash suggests (an inference, not observed): `malloc` had just handed the block out (every path of newlib's `malloc` writes or uses the size word before it returns), so the word was overwritten — with a pointer to the block's own neighbourhood, as the `bk` link of a free chunk 16 bytes lower would be — between `malloc`'s return and Skia's next instruction or by `malloc`'s own unlinking of a damaged bin: a free list already damaged (a write after free, a block freed twice, an overflow into a freed neighbour), not a size reported wrong. Earlier `HEAPCHECK=1` runs do not rule that out: they kept the workaround, so Skia never used its slack under the canaries. To resume: `SKMALLOCSIZE=0 sh tools/webkit/build-web.sh` (Web as it crashed) and `HEAPCHECK=2 sh tools/webkit/build-web.sh` (heapcheck with blocks as large as newlib's and a `malloc_usable_size` that says so, Skia filling them: an overflow of the usable size, a write after free or a double free is then reported with the callers that allocated and freed the block). Fixed in `heapcheck.c` on the way: the program's first blocks (the first 4 KB of the heap) were taken for newlib's own and freed to it by their inner address.
- On the bench: a reused socket number was taken for a connection's own peer (posixsim's fake kapi).
- `/tmp` is `RAM:/tmp` for an Onyx program (libonyxposix): test pages live in `SD:/wktest`.

Found and **not fixed** (Onyx, outside WebKit):

- **A kernel panic in Circle's TCP** (`tcp: Unexpected state 0 at line 1922`, `SD:/etc/lastcrash.txt`) when a telnet client closed its connection while `kmsg` was writing to it: a network client can restart the Pi. Probably also the restart seen while `cat` joined a big file over telnet.
- `ps` shows absurd `PAGES` numbers (2485159040) for the web processes (with shared images).

Then (2026-10-03, patch `0016`), tried on the Pi over VNC: the pages' **drop-down lists** (the C API's `WKViewClientV1` `showPopupMenu` / `hidePopupMenu` and `WKViewSelectPopupMenuItem`, from `UIProcess/onyx/WebPopupMenuProxyOnyx`; Web shows the list in a uikit popup and answers — the `change` event fires), the **system clipboard** both ways (`WKSetClipboardCallbacksOnyx`: WebCore's `PlatformPasteboard` of the UI process calls Web's three functions, `clipboard.h` underneath; the system's text is read again when its serial changed), **find in the page** (`WKPageFindString`, the matches counted and marked), the menu of the right button and **downloads**: a response the page cannot show, or an attachment, becomes a download (`WKFramePolicyListenerDownload`), *Save …* *As* starts one (`WKPageDownloadURLOnyx`: the page's Referer and User-Agent, its cookies); the file goes to `SD:/Downloads` (Web makes the folder and a unique name: WebKit opens it `O_EXCL`), the progress to the downloads' window — the same program run with `--onyx-downloads`, its stdin / stdout two pipes. Two fixes on the way: curl's download path told WebKit no running total nor expected length (no progress could be shown: `NetworkDataTaskCurl`, in `0016`), and libonyxposix answered `EPIPE` to a write into a pipe whose read end it had given as a child's stdin and then closed (its counts are the process's own: `user/Runtime/libc/posix/proc.c`, docs/03 §5.4). A third, 2026-10-04: the downloads' window (and the console's) froze once the browser had nothing more to tell it — after a Cancel, its close button did nothing: `read (0)` waited although the descriptor was `O_NONBLOCK` (libonyxposix's console descriptor ignored the flag: `user/Runtime/libc/posix/file.c`). Web also serves Mail as its HTML view (`--applet`, `user/Apps/jet/webview.cpp`; docs/03 *Web's web view*).

Not there yet (the next work of step 1): the pointer's shape (no kapi for it), the window's title in the dock (the kernel keeps the program's), WOFF2 fonts, the measurements of where kotonstudio.com's 3.2 s go.

The JIT (roadmap step 3, 2026-10-03)

What it needed from Onyx: executable memory. Kernel v78 gives `PROT_EXEC` to anonymous regions (lazy pages mapped `UXN = 0`; `vm_protect` sets or takes it away: W^X possible; docs/02 *v78: PROT_EXEC*); libonyxposix's `mmap` / `mprotect` pass the kernel's answer on; the caches are made coherent from EL0 (`__builtin___clear_cache`: `DC CVAU` / `IC IVAU`, allowed since SCTLR_EL1.UCI / UCT were set for the GameCube emulator). JavaScriptCore's executable allocator reserves its pool `RWX` as on Linux (its default there): nothing in WebKit had to change for it — the port already had the `OS(ONYX)` cache flush, and no signal-based VM traps (`HAVE(MACHINE_CONTEXT)` is off: the traps are polled). One upstream build fix: WebAssembly's `AddressType` used B3's types whenever the JIT was on, but B3 exists only with the FTL (`0017`).

The build: `INTERP=jit sh tools/webkit/build-jsc.sh` (`-DENABLE_JIT=ON -DENABLE_DFG_JIT=ON`; not yet the FTL — B3 — nor WebAssembly's BBQ / OMG JITs; WebAssembly keeps its interpreter).

The tests (`INTERP=jit sh tools/webkit/test-jsc.sh smoke es6 stress wasm`, the PC bench): smoke ok; es6 595 passed, the 10 expected failures; stress (one test in 10) 469 / 469; wasm 594 passed, the 17 expected failures — the same results as the LLInt build. On the Pi: smoke ok.

On the Pi (`bench.js`, the LLInt build → the JIT build, ms): fib(30) 2040 → 351 (×5.8); 20M additions 5533 → 706 (×7.8); 1M allocations 2410 → 919; 300k-number sort 4902 → 1781; 100k regexp 1405 → 563; a

Map's 500k set / get 4211 → 2199; 5M typed-array doubles 17981 → 493 (×36); strings 2374 → 2525 (the same: C++). *Open*: the JSON case of `bench.js` is slower with the DFG in that script (983 → about 8000 ms) and normal without OSR entry into the DFG (`--useOSREntryToDFG=false`: 1187 ms) or alone (`json.js`-like scripts: the same as the LLInt) — to understand.

The browser with the JIT (Web 1.0.3): a second WebKit tree, `~/webkit-build/webkit-jit` (`BUILD=… CMAKE_EXTRA="-DONYX_WEBKIT=ON -DENABLE_JIT=ON -DENABLE_DFG_JIT=ON -DENABLE_FTL_JIT=OFF" TARGET=WebKit sh tools/webkit/build-webcore.sh`: 44 minutes on 16 cores), and `BUILD=… sh tools/webkit/build-web.sh` (102 MB). On the Pi: kotonstudio.com as before (about 3.5 s: its time is the network's and the painting's), **a GitHub repository's page in 5.4 s from the launch to the load's end** (about 50 s in the interpreter). Seen once, not again: 57 s before the first request of a launch that followed a browser killed while loading (a lock left in `SD:/var/webkit`? to watch).

Next: the FTL (B3 on ARM64) and WebAssembly's BBQ when the browser needs them.

The compositor on the V3D (roadmap step 2, begun 2026-10-03)

The route. In the pinned revision WebKit's own compositor (Coordinated Graphics) is tied to GL and to Skia's Ganesh from end to end (its tiles are GL textures, its compositor a GL context: `ThreadedCompositor`, `AcceleratedSurface`, `SkiaCompositingLayer`), and TextureMapper is GLSL. So the port has **its own `GraphicsLayer` on Onyx's `user/Libs/gpucomp`** (the model: Windows' `GraphicsLayerWC` and its tile grid), as Jet does (the removed docs/06) — patch `0018`:

- `WebCore/platform/graphics/onyx/GraphicsLayerOnyx`: a layer's properties and a grid of 512-pixel tiles kept over a window around what is seen (one view above, two below), painted by Skia on the CPU.
- `WebKit/WebProcess/WebPage/CoordinatedGraphics/LayerTreeHostOnyx` (the web process): flushes the tree, paints the new or dirty tiles, hands them to gpucomp as textures (`gpc_tex_create` / `gpc_tex_update`), walks the tree into a list of `gpc_layer` (translations, rectangular clips, opacity) and composites **straight into a kernel surface** (`surface_create`: below 1 GB, so the V3D stores its tiles there without a copy) that the UI process made, sized as the screen, and maps; each frame is told to the UI process (`DrawingAreaProxy::OnyxCompositedFrame`: size, damage), which copies what changed into the window's canvas (`WKPagePaint`). One composite a display refresh at most.
- gpucomp is compiled into the browser's link (`build-web.sh`, with `tools/webkit/onyxsurface.c`, the kapi calls behind `Shared/onyx/OnyxSurface.h`).
- **The default since Web 1.0.6** (the user, 2026-10-03; it was `web --gpu` or the file `SD:/etc/web-gpu` before): `web --nogpu`, or the file `SD:/etc/web-nogpu`, keeps the software path, which is unchanged (`WKPreferencesSetCompositingEnabledOnyx` before the view is made); Mail's web view (`--applet`) stays on the software path. Test switches: `SD:/etc/web-gpu-cpu` (gpucomp on the CPU), `SD:/etc/web-gpu-copy` (composite into a `gpc_target_alloc` buffer, then copy), `SD:/etc/web-gpu-debug` (each layer's paints in kmsg).

Stage 1 on the Pi (kotonstudio.com, a 1000 × 638 view): `gpu: compositing on, backend GPU: V3D 4.2`, `target path: straight into the surface`; the picture and the scrolling are right; a composite takes about 0.65 ms; `gpu: 2.0 s: N frames, M tiles painted, paint / upload / composite ms` every two seconds. What it shows: only the root is composited at this stage, so the page's CSS animation repaints a tile at every frame (4.4 ms), and its reveal-on-scroll effects repaint tiles while scrolling (35 ms a tile) — **stage 2: layers for animated opacity / transform, composited without a paint**, the partial repaint of a

tile, groups for opacity. On the bench (`WK2TEST_GPU=1 sh tools/webkit/test-webkit.sh`, gpucomp on the CPU): the 49 checks pass on both paths, a scrolled page matches the software picture.

Stage 2 (patch `0019`): layers for CSS animations and transitions of **opacity and transform** (the `Animation` and `AnimatedOpacity` compositing triggers); `addAnimation` is taken and the animation evaluated at composite time (WebCore's `TextureMapperAnimation`, GL-free), so an animated frame paints nothing and needs no page update; a layer drawn scaled, rotated or at a fractional place is one bilinear texture (≤ 2048); **opacity groups** (overlapping composited descendants: a gpucomp alpha target uploaded again as a texture); a budget (440 textures, 96 MB); a dirty tile repainted only where it is dirty. And three fixes in WebCore that scrolling needed:

- a page's **sticky** elements are layers when the frame is composited (without a scrolling coordinator they were not);
- a composited fixed or sticky element is **not laid out again** when the layout viewport moves (its layer's place is updated): each scroll step laid them all out and repainted their blocks;
- `RenderLayer::paintList` skips a child subtree whose repaint rectangles do not touch what is painted (a 300-pixel repaint walked every layer of the page). Switches, files in `SD:/etc`: `web-gpu-debug` (each layer's paints, a `times:` line), `web-gpu-trace`, `web-gpu-noanim`, `web-gpu-nogroups`, `web-gpu-nocull`, `web-gpu-3d`, `web-gpu-stage1`. The bench: 65 checks on both paths (animations paused: the software picture within 2 levels; running: 0 tiles painted; a page with fixed and sticky elements scrolled: 0 rectangles repainted).

Stage 2 on the Pi (the V3D, a 1000 × 638 view):

- Wikipedia's Raspberry Pi article, 30 notches of the wheel: stage 1 repainted the whole view at every step (13.5 Mpx in 2 s, 16 frames); now 36 frames in 2.1 s for 580 ms of work in all (page update 280 ms, the layers 300 ms, of it 218 ms of painting: the new tiles) — the compositor is no longer what limits the scrolling.
- kotonstudio.com: its reveal effects are composited (each card a layer painted once); what is left is the page's own painting — blurred shadows, a large scaled picture: 0.2 to 0.7 ms a thousand pixels in Skia on the CPU, as in the software path — when a card's layer appears and goes (the parent is repainted there), and 4.4 ms for each repaint of its pulsing dot (a `box-shadow` animation: not a composited property).
- *To do*: the window's canvas shared with the web process (a kernel addition: no copy at all), 4 × 4 matrices in gpucomp (3D), composited filters and masks, a compositor thread.

The tiles rasterised on the other cores (patch `0020`, Web 1.0.5). On Onyx every POSIX thread runs on core 0: another core is an **app core** (`kapi_core_acquire` / `core_run`, docs/03 *App cores*: cores 2 and 3, no kernel call from there). So the compositor paints in two passes — the main thread **records** each dirty rectangle once into an `SkPicture`, then jobs of at most 512 × 256 pixels **replay** it into the tiles' buffers on up to two app cores and on the main thread (pure Skia there: no WTF, no WebCore), the uploads and the composite staying on the main thread. Under 64k pixels, or without a free core, the direct path. `tools/webkit/onyxcores.c` (linked by `build-web.sh`; `Shared/onyx/OnyxCores.h`): the workers with a TLS block of their own, libonyxposix's RPC served while a batch runs (`sbrk` — malloc's growth —, file reads: `pread` / `pwrite` wrapped onto it), core 0 never asleep on a lock an app core holds (`--wrap` of newlib's lock, `sem_wait`, `pthread_mutex_lock`: try, yield, retry), the watch on a core that faults or stalls (its jobs done again on the main thread, the process then single-threaded; a worker dead with a lock held: the web process exits, WebKit starts another). The cores are given back after 10 s without raster (the emulators want them). Switches: `SD:/etc/web-gpu-nocores`, `web-gpu-mainwaits`, `web-gpu-coretest` (eleven stages

on a core — arithmetic, malloc, `new`, thread_local, a static's guard, Skia's shapes, gradients and blur, text, a scaled image, a picture replayed —, then the page's first 200 jobs one at a time; lines `gpu: core test …`). The bench has real concurrent cores now (`POSIXSIM_CORES=2`: host threads that answer a non-zero core and **fail the test on any kernel call**).

- **On the Pi** (kotonstudio.com, one app core free + the main thread): the 12-notch scroll's worst two seconds went from `20 frames, 7 Mpx, paint 1450 ms` to `64–73 frames, 5–7 Mpx, paint 200–340 ms`; the self-test's stages all pass.
- **What this found in Web's link**: `user/Kits/imagekit/img/imgload.hpp` (in uikit) defines *weak* `malloc` / `free` / `calloc` / `realloc` on `operator new[]`, and `onyxpp.hpp` a global `operator new` on `umm.h` (`kapi_sbrk`): in the static link both won over newlib's. So the whole browser — Skia, WebKit — allocated with uikit's allocator, while `malloc_usable_size` and `posix_memalign` were newlib's: **the heap corruption in Skia's glyph painting of Step 3** (a usable size read from a block newlib did not make) has its cause — `skmallocsize.c` was the cure of a symptom, kept. A hosted program now defines `ONYX_HOSTED_NEW` (`build-web.sh`): newlib's malloc everywhere, checked at the link (no weak `malloc`, no `umm__*`).

Media: video and sound (roadmap step 4, begun 2026-10-03)

`ENABLE_VIDEO` and `ENABLE_MEDIA_SOURCE` are on (`OptionsOnyx.cmake`; Web Audio is not yet). What made it urgent: **YouTube's application stops at its first line without `HTMLVideoElement`** (`ReferenceError: Can't find variable: HTMLVideoElement` in its main script — the page stays its grey skeleton); with the interface there the pages draw, and the videos play.

The engine is Onyx's own media library (`user/Libs/av`, docs/03 *The media library*: the demuxers, libvpx / dav1d / opus, a player with the sound as the clock), not GStreamer:

- `platform/graphics/onyx/MediaPlayerPrivateOnyx.{h,cpp}` — one class for the two ways a page gives media, registered in `MediaPlayer.cpp` (`MediaEngineIdentifier::Onyx`). It owns an `av_player`, polls it from a main-thread timer (12 ms while pictures may be due, 60 ms otherwise): the state (ready, time, duration, ended, a seek's end), and the picture due now — the player's 32-bit pixels (B, G, R, A: Skia's N32 in Onyx's build) copied into an `SkImage`, painted by `paint()` (the software path: the compositor takes it in the element's tiles).
  - **A file** (`src=`): fetched through the page's `PlatformMediaResourceLoader` and fed to the player's store where its demuxer wants bytes (`av_store_feed` / `av_store_want`), with Range requests: an MP4's index at its end, a seek, and the request stopped 30 s of media ahead of the playback position, started again under 15 s. The container is read from the first 4 KB (`av_probe`), not from the server's type.
  - **Media Source** (`MediaSourcePrivateOnyx.{h,cpp}`: `MediaSourcePrivateOnyx`, `SourceBufferPrivateOnyx`): WebCore does MSE's bookkeeping (`SourceBufferPrivate`, `TrackBuffer`: the coded frames, the buffered ranges, removal, eviction). The buffer parses what is appended with one of the library's demuxers (`av_demux_append` / `av_demux_read`; an initialization segment → the tracks told to WebCore), wraps each packet in a `MediaSample`, and puts the samples WebCore *enqueues* (in decode order, about 3 s ahead of the playback position: `isReadyForMoreSamples`) into **a queue of the player's store** — `av_store_add_queue`, a source without a parser, added to the library for this (`av_store_queue_track / _put / _flush / _end / _level`; its test: `avtest.c`'s `test_player_queue`). A seek: `waitForTarget`, the queues flushed and filled again from the random access point (`reenqueueMediaForTime`), then `av_player_seek`.

- **What plays**: VP8, VP9, AV1, Opus, FLAC, MP3, PCM, in WebM, MP4 (fragmented too), WAV, FLAC, MP3 files. **No H.264, no AAC**: the vendored FFmpeg is a GPL build (the Media Player's); Web is LGPL and stays so (`docs/LICENSING.md`) — an LGPL build of FFmpeg's decoders is the way, if wanted. YouTube serves VP9 + Opus when H.264 is refused. `canPlayType`, `MediaSource.isTypeSupported` and `navigator.mediaCapabilities.decodingInfo` (`smooth`) answer with `av_type_supported`: the sizes the Pi decodes in real time (VP9 up to 480p), so that a player picks a stream it can show.
- **The sound** is the kernel's output (one owner: the first player that plays is heard, the others run silent on the wall clock). **The video decoder runs on an app core** when the system has one free (core 2; the network has core 3): `onyxcores.c` has a *side job* beside the raster batches (`onyx_cores_offload`: one job, taken by a worker before a batch's next job, its poster waiting) and the library a hook for it (`av_set_offload`: a packet decoded and its picture converted in one job); without a core (an emulator runs), on the player's own thread, as before.
- **Linked**: `tools/webkit/av.mk` builds `libonyxav.a` (user/Libs/av, `-DAV_POSIX -DAV_KAPI_SOUND`, with libvpx, dav1d, opus) for the POSIX toolchain; `build-web.sh` adds it.
- **Tests**: `tools/webkit/tests/video-file.html` and `video-mse.html` (made by `mkvideotests.py` from `tools/tests/av/clips`): each ends with `video test: PASS` in the console. The engine's lines in `kmsg`: `web: media: ...` (a source buffer's tracks; every 5 s, the time, the size, pictures decoded / dropped, the time a picture takes, how many went to the app core).
- **The pictures are a layer of the compositor** (patch `0024`). Painted into the page's tiles — the element invalidated for each picture, the picture scaled by Skia on the main thread —, a 480p picture cost ~190 ms and five in six were dropped. Now the player owns a `GraphicsLayerOnyxContents` (`GraphicsLayerOnyx.h`; it is the port's `PlatformLayer`: `platformLayer()`, `supportsAcceleratedRendering()`), a video element is a layer (`ChromeClient::VideoTrigger` in `onyxCompositingTriggers`), and `RenderLayerBacking` gives the contents to the element's `GraphicsLayerOnyx` (`setContentsToPlatformLayer`). For each picture the player calls `setPicture` (its pixels, not copied) and the host schedules a frame *without* the page's update; the frame's walk copies the pixels into the layer's own texture (`LayerTreeHost::prepareContents`: `gpc_tex_update`, the texture made again when the size changes), damages the contents rectangle only, and draws the texture scaled to it over the layer's own content. Nothing is painted: `web: gpu:` lines say `N video pictures`, `0 tiles painted`. Without the compositor (`web --nogpu`, Mail's view) no layer takes the contents and the player paints as before (an image is made of the pixels only then: `ensureImage`). `SD:/etc/web-gpu-novideo`: no video layers. **Measured on the Pi 4**: YouTube, VP9 480p: 59–61 pictures shown every 2 s, one dropped every few seconds (84 % were); the two test pages: no picture dropped.
- **Not there**: Web Audio, the picture as a YUV texture (it is converted to 32 bits on the app core), full screen, H.264 / AAC, captions.

A site's own user agent (Web 1.0.8, patch `0025`). YouTube's and Google's desktop pages are heavy for a Pi 4; the browser asks for them as Android's Chrome (`engine_webkit.cpp`: `site_user_agent`, `apply_site_user_agent` — `WKPageSetCustomUserAgent` where a typed address is loaded, and in the navigation policy for the main frame: when the user agent changes there, a plain GET navigation is started again so that its request carries the new one). Not an iPhone's: it would be given HLS / H.264, which the media engine does not play; Chrome on Android gets Media Source with VP9. YouTube then serves `m.youtube.com` (loaded in 6.4 s; the desktop page: 12–14 s). The port's `standardUserAgentForURL` returns an empty string: it returned the standard user agent for every URL, and `WebPage::userAgent()` takes that before the page's own — the custom user agent was never sent. Switches: `SD:/etc/web-desktop-ua` (none), `SD:/etc/web-mobile-ua` (every site).

The console. WebKit's Onyx port sends every console message to the UI process (`WebPageProxy::OnyxConsoleMessage`, from `WebChromeClient::addMessageToConsole`; the console API's messages with all their arguments: `FrameConsoleClient.cpp`), and the embedder gets them through `WKSetConsoleMessageCallbackOnyx` (`WKPagePrivateOnyx.h`). Web keeps the last 1000 and shows them in its Console window (View ▸ Console, F12: `user/Apps/jet/console.cpp`, a process of its own as the downloads' window; emptied at each navigation). Two files for whoever debugs a site: `SD:/etc/web-console` (the console in `kmsg` too) and `SD:/etc/web-probe.js` (a script run in every page before its own: `tools/webkit/tests/probe.js` reports errors, failed resources and the document's state — how the `HTMLVideoElement` error was found).

A finding on the way: every program ran four times slower. `clipd` — the clipboard's service — waits for its messages with `kapi_mailbox_recv (..., blocking)`, and the kernel's blocking receive was a loop of `Yield`: the receiver stayed ready and took a turn of core 0 at every round. On the Pi `ps` showed it `R` with no system call, and `tools/webkit/tests/mbench.c` (malloc, sbrk, page faults timed as the heap grows) showed steps of exactly 60 ms in what takes 13: about 80 % of core 0 gone. The receive now sleeps on the I/O generation (`IoWait`; `CMailbox::Push` calls `IoWake`: `kernel/sys/ipc.cpp`; onyx 2026.10.35). With core 0 free, `jsc`'s loops ran six to seven times faster on the Pi (a 3 M loop: 33 ms for 216).

And the collector. JavaScriptCore's collector is concurrent by default: it marks beside the script and, to keep up, takes the processor from it. On Onyx a program's threads share one core: `tools/webkit/tests/alloc.js` in `jsc` on the Pi kept 400 000 short strings in 3.9 s, and in 0.2 s with `--useConcurrentGC=0` (200 000 objects: 758 ms against 20; `--logGC=1` showed the mutator down to 2 % of the time, a cycle of 3.6 s). The port's defaults are now `useConcurrentGC = false` and `numberOfGCMarkers = 1` (`Options.cpp`, `overrideDefaults`; patch `0023`; a command line can ask for them again): the collection is generational and stops the world for its length. The other scripts of that folder: `strhash.js` (is a string-keyed table linear), `strget.js` (its reads), `sched.html` (the page's event loop: tasks, timers, frames, and the same loops).

The roadmap from here (the user, 2026-10-02)

In this order; a later step is not started early:

1. **A usable browser** — the target: **kotonstudio.com near-instant on the Pi**. This is WebKit2 (the UI, web and network processes over Onyx's IPC), the Onyx view and the browser's UI, pages loaded over HTTPS. The criterion is the time to show the page once the browser is open. **One executable for the three roles** (the user's decision): the UI, the web and the network process are the same program, its role chosen at launch. **This step includes the kernel's shared read-only image and the preload list** (the user's green light, 2026-10-02: `docs/ELF-LOADER-PLAN.md`, stages a, c and e — `preload` lines in `/etc/autostart`, `/bin/preload`, `/bin/unload`), which is what makes the single executable pay — three unshared 80 MB process images would take about 15 s to load. It is done on its own branch (`shared-image`, from `main`) and tested on the Pi before it is merged.
2. **The compositor on the V3D.**
3. **The JIT** (executable memory in the kernel first).
4. **Video** (`MediaPlayerPrivate` on `user/Libs/av`, MSE).
5. **The host window + the web view that attaches to it**, a component the Mail client embeds too. Asked by the user for this step or after it (2026-10-02): **a web-view daemon started at boot**, idle until a window asks for a web view, which then starts the UI, web and network roles for that

window. What it buys: it holds the program's shared image (ELF-LOADER-PLAN stage c), so the 5 s SD read is paid once, in the background after the desktop is up, and every web view after that starts without reading the card; it is also the natural owner of one network process shared by all the views (one cookie jar, one connection pool, one cache) and of a spare, already initialised web process (WebKit's process prewarming) handed to the next view. Its price: about 80 MB held from boot — an option to turn off on a 1 GB Pi, where stage d (the image kept after exit) gives the same from the second start on. **A `fork` is to be implemented in the kernel for this** (the user, 2026-10-02), so that the daemon is a zygote: it initialises the engine once (WTF, ICU, the font list, Skia) and each role is a clone of it instead of a fresh start. What that fork needs: the read-only image shared (stage c), the writable data, the heap and the calling thread's stack copied — copy-on-write of 64 KB pages, or an eager copy while those are small —, the descriptors duplicated, a kapi addition (append-only). Its constraint, as on every system: only the calling thread exists in the child, so the zygote must fork **before it starts any thread** (the collector, the timers, the IPC monitors are started by the child), and locks held at that moment must not exist. To measure first: how much of a role's start-up is initialisation that a zygote can do ahead, next to the image load that stage c already removes.

6. **Lazy loading** (the image filled page by page from the file: `docs/ELF-LOADER-PLAN.md`, stage b).

WebGL comes after these (its route is chosen: step 2 of the WebCore section above).

The plan of the whole port

WebKit replaces Jet (NetSurf) as Onyx's browser engine (docs/POSIX-PLAN.md §8, §9, §15: WebKit2 on the PlayStation port's model, static binaries, distributed under LGPL-2.1+):

1. **WTF + JavaScriptCore** → the `jsc` shell (the LLInt without JIT, WebAssembly in its interpreter; C_LOOP as an alternative). *Done: validated on the Pi, 2026-10-02.*
2. **WebCore** (Skia CPU raster from WebKit's own copy, no GL, `SkFontMgr_onyx` instead of fontconfig, curl + mbedTLS networking, ICU, HarfBuzz, libxml2, SQLite, woff2). *Builds, links, renders a page on the bench and on the Pi (2026-10-02); WebCrypto on mbedTLS passes its tests on the bench; the network path, WOFF2 and media are still to do.*
3. **WebKit2** (UI, web and network processes over WP-IPC: AF_UNIX socketpairs, SCM_RIGHTS, shm). *Done and running on the Pi (2026-10-03): patches `0013`–`0015` (see "Step 3" below), the browser **`Web`** (`user/Apps/jet`, package `web`).*
4. The Onyx view; then, in the order the user asked for (2026-10-02): **the JavaScript JIT before video**. The JIT: executable memory for a program's own mappings in the kernel (`PROT_EXEC` through `vm_map` / `vm_protect`, W^X, the instruction cache flushed from EL0: a kapi change), then `ENABLE_JIT` (the Baseline tier first) with JavaScriptCore's executable allocator on it — and a check of what the JIT tiers ask of thread suspension and signals, which Onyx lacks (above). Then media (`MediaPlayerPrivate` on `user/Libs/av`, MSE: Jet's decoders — VP9, AV1, Opus — are there), and the compositor on the V3D once the CPU rendering has proved itself. YouTube is the reference site: its pages need the JIT to feel fast, its videos need MSE.
5. The browser (Jet's UI reused). **No tabs** (the user, 2026-10-02): one page per browser window — one UI, one web and one network process per browser; what `window.open` and `target=_blank` do is to be decided then (the same view, or a new window). **Start-up** (the user, 2026-10-02): the large images are slow to load (above). Two ways, which add up: a lazy, file-backed ELF loader in the kernel (pages read from the file when first touched, code pages shared between the processes: the study is `docs/ELF-LOADER-PLAN.md` — the time is the SD read at 16 MB/s, the images are 99.8 % read-only, and sharing one loaded image between the processes of one

executable is the cheap, safe gain; lazy fill is riskier and only helps the first start), and — the fallback if that cannot be done, and good for the perceived speed in any case — **a small front end, the host window, separate from the web view** (the page and the toolbar): the front end is a few hundred KB, its window appears at once, and it starts the web view, which then attaches to the window as an applet does (to check: how a process draws into another's window on Onyx — an existing applet mechanism, or a canvas in v76 shared memory — and how the key and mouse events reach it). If both are feasible, both are wanted, and **the web view is then a component other programs embed — the Mail client first** (HTML messages): a stand-alone web-view service with a small host interface (load a URL or the HTML given, back / forward / reload, the title and URL changes, a hook on navigations), an optional toolbar (the browser's; Mail has none), and restrictions the host sets (Mail: no remote content until the user allows it, scripts off, a clicked link opened in the browser). The browser's own UI does not go into the web view.

6. Parity with Jet, then the switch.

The patch series

`tools/webkit/patches/*.patch` (git format-patch; each patch's message says what and why), applied by `tools/webkit/fetch.sh` with `git am` onto the pinned revision. WebKit is not vendored in the Onyx repository. `/bin/jsc` is distributed under LGPL-2.1+ (docs/LICENSING.md): the pinned revision, this series and `build-jsc.sh` are its complete corresponding source.